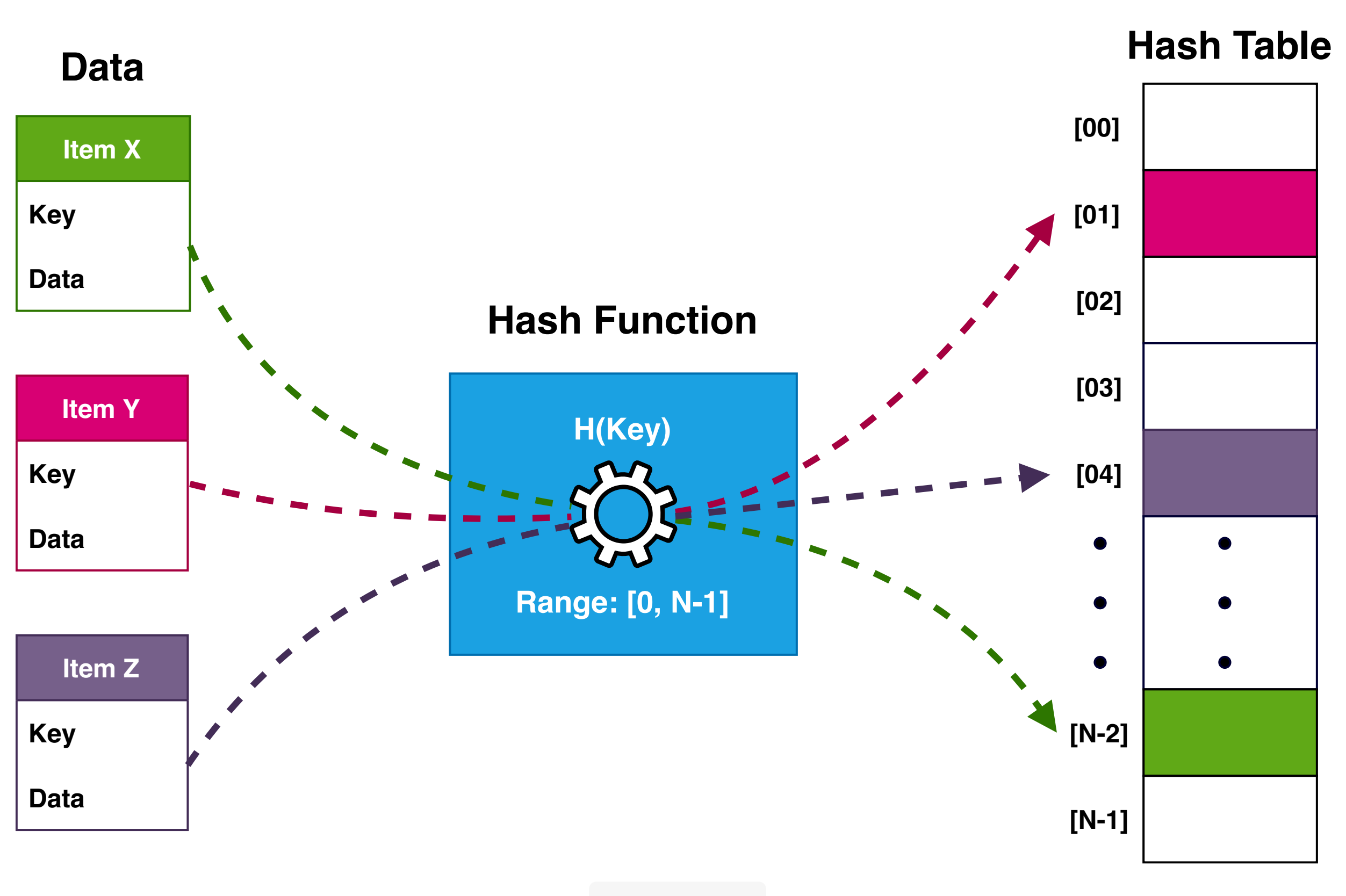
Introduction to Hash Tables

A Hash Table (also known as Hash Map), at its core, is a data structure that allows us to store and retrieve data efficiently. If we think about a real-life analogy, it's like a library where each book (data) has a unique identifier (key) like ISBN, and all books are organized in a specific way to allow the librarian (hash function) to find and retrieve them quickly.

In other words, a Hash Table implements an associative array abstract data type (or Dictionary ADT), mapping keys to values. A Hash Table uses a hash function to compute an index into an array of buckets or slots where the desired value is stored.



Hash Table

There are four main elements to any Hash Table: Keys, Values, the Hash Function, and Buckets.

- **Keys**: In our library analogy, think of keys as the unique identifiers for each book. Keys are the inputs we feed into our hash function. They can be any data type - numbers, strings, or even objects. The crucial characteristic of keys is that they should be unique. If two pieces of data share the same key, it might lead to complications, like collisions (we'll discuss this in detail later).
- **Values**: Values are the actual data that we store in our Hash Table. They could be anything from a single number to a complex object or even a function. Using the key, we can quickly retrieve the corresponding value from the Hash Table.
- **Hash Function - H(x)**: We've touched on this before, but it's worth emphasizing the importance of the hash function. This is the engine that drives a Hash Table. It's responsible for transforming keys into hash values, which dictate where we store our data in the table.
- **Buckets**: Once the hash function processes our key, it produces a hash value. This value corresponds to a specific location or 'bucket' within the Hash Table. Think of buckets as shelves in our library, each one designated to store a specific book (or piece of data).

Here are the three basic operations that are performed on Hash Tables:

- **Insert(key, value) operation**: Calculates the hash index $H(key)$ and stores the key-value pair in the slot at that index.
- **Search(key) operation**: Computes $H(key)$ to find the slot and returns the value associated with the key, or `null` if not present.
- **Delete(key) operation**: Removes the key-value pair stored at index $H(key)$.

A naive Hash Table implementation

Imagine once again a major public library that needs to store basic information, such as Key (ISBN), Title, and Placement Info, for all available books. This system is heavily used, with librarians frequently searching for books. This results in many retrieval requests.

The frequent retrievals require an efficient solution that can quickly perform the search operation (ideally, in constant time). Therefore, the Hash table data structure perfectly suits the scenario.

Let's start by discussing our data model. We will use a dynamic array of the `Record` type to store the books' information. Here is what our `Record` class looks like:

Java

```java
public class Record {

    public int key;
    public String title;
    public String placementInfo;

    public Record() {
        Key = -1;
    }

    public Record(int key, String title, String placementInfo) {
        key = key;
        title = title;
        placementInfo = placementInfo;
    }
}
```

ADT class

Now, let's look at the `HashTable` ADT class definition. The `HashTable` class has an HT_array pointer to store the address of the dynamically allocated array of records. The `max_length` property is the maximum number of records the hash table can hold. The `length` represents the current number of records in the Hash table. It increments and decrements with insertions and deletions, respectively.

Java

```java
public class HashTable {

    private Record[] htArray; // Array to store records
    private int maxLength; // To store maximum number of elements a Hash table can store
    private int length; // To keep track of total records present in the hash table

    // The Hash function
    private int hash(int key) {
        // Implement your hash function logic here
        // Return the calculated hash value (index within the range of the hash table's
        // Example: return key % max_length;
        return 0; // Placeholder, implement your hash function
    }

    // Constructor to initialize the HashTable
    public HashTable(int size) {
        maxLength = size;
        length = 0;
        htArray = new Record[size];
    }

```

Defining the hash method

Let's define our **Hash method** `H()` for this naive implementation. We will use the simplest modular Hashing for this scenario:

Java

```java
public int hash(int key) {
    return key % max_length;
}
```

The above Hash function ensures we always get an index value in the range $[0, max\_length-1]$. Let's move on to see how the simplified/ naive `Insert()` method works.

Naive insertion operation

The `Insert()` method takes a new record as a parameter and checks if the maximum capacity of the `HT_array` is not reached. If the table still can have more records, the method calculates a proper index/ hash key for placing this item. After that, it puts the item at the computed index.

Java

```java
blic boolean insert(Record item) {
  if (length == maxLength) {
      System.err.println("Hash table is full. Cannot insert the key-value pair.");
      return false;
  }

  int index = hash(item.Key);
  htArray[index] = item;
  length++;
  return true;


```

Point to ponder: What happens if two different keys map to the same array index? This implementation overwrites it. Indeed, it is a flaw we will address in the Solving Collisions section.

Naive search operation

Now, let's explore how and why the **Search**() method will retrieve records in $O(1)$ for us. Here is a naive implementation:

Java

```java
public boolean search(int key, Record returnedItem) {
    int index = hash(key);

    if (htArray[index].key == -1) {
        return false; // Record not found
    }
    returnedItem.key = htArray[index].key;
    returnedItem.title = htArray[index].title;
    returnedItem.placementInfo = htArray[index].placementInfo;
    return true; // Return true to indicate the record was found
}
```

The `Search()` method applies the hash function `H()` on the passed key and checks if the hash table slot is empty or not. If this slot has a valid record, the `Search()` method assigns the record at this slot to the reference parameter. Also, it returns a `true` flag to indicate the operation's success.

The above implementation of `Search()` clearly takes constant time (i.e., $O(1)$) time to retrieve/ search any record regardless of the size our table may grow. This is evident by the fact that you only have to apply the hash function only constant number of times to calculate the position of the searched item.

Note: This naive implementation for the search doesn't cover all aspects. We will discuss a more sophisticated method in the next lesson.

Naive delete operation

Like the search operation, this deletion operation first locates the item requiring a delete. Afterward, the deletion operation simply places a null or default object at the table slot.

Here is what the naive implementation looks like:

Java

```java
public boolean delete(int key) {
    int index = hash(key);

    if (htArray[index].key == key) {
        htArray[index].key = -1; // Mark the slot as empty
        length--;
        return true;
    }
    return false; // The slot is already empty or there is a different item at the slot
}

```

Again, like the search operation, deletion also takes a constant time to locate and delete an item from the table. But, it is important to note that this naive implementation is just to get an overall idea of how the hash table works. However, it doesn't cater to many exceptional cases; we will discuss those in the subsequent sections.

Here is the complete code for the naive hash table implementation with a sample driver code:

Java

```java
import java.util.Arrays;

class Record {
    public int key;
    public String title;
    public String placementInfo;

    public Record() {
        key = -1;
    }

    public Record(int key, String title, String placementInfo) {
        this.key = key;
        this.title = title;
        this.placementInfo = placementInfo;
    }
}

class HashTable {
    private Record[] htArray;
    private int maxLength;
```

Run Code Result
Stdout

You must run your code first

Execute

Implementation

Here is how different languages have implemented Hash Tables:

Language	API
Java	java.util.Map Or java.util.HashMap
Python	dict
C++	std::unordered_map
JavaScript	Object or Map

← Previous
Introduction To Hashing

Next →
Issues With Hash Tables

☐ Mark as Completed